



## CLOUD DASHBOARD DEVELOPMENT

# Project Phase 2 Diet Cloud Dashboard

Azure Functions, private Blob Storage, interactive data visualizations, and repeatable cloud deployment

## PROJECT TEAM

**Asad Arif**

**Can Ozveren**

**Muhammad Talha Arif**

Technical implementation report | July 17, 2026

Verified locally and in Azure against the complete 7,806-row Phase 1 dataset. Live deployment evidence and public links are included in the final section.

---

# Executive summary

Phase 2 converts the Phase 1 local analysis into a cloud-native, browser-accessible dashboard. A Python Azure Function reads the Diets Dataset from a private Blob Storage container, performs cleaning and aggregation, and returns compact JSON for four interactive visualizations. Azure Static Web Apps hosts the responsive frontend, while GitHub Actions supports continuous testing and separate frontend/backend deployments.

## Implementation status

| Area         | Status   | Evidence                                                                                           |
|--------------|----------|----------------------------------------------------------------------------------------------------|
| Backend API  | Complete | Filterable Azure Functions v2 endpoint with validation, caching, metadata, CORS, and errors        |
| Dashboard    | Complete | Four charts, five KPIs, dynamic filters, loading/error states, accessible table, responsive layout |
| Cloud assets | Complete | Flex Consumption Bicep, private storage containers, monitoring, CORS, TLS, deployment scripts      |
| Verification | Complete | 4 unit tests; live 7,806-record API; filter/reset and responsive-browser checks; successful CI/CD  |
| Azure links  | Complete | Function, Static Web App, and GitHub URLs verified; deployment evidence included                   |

### DEPLOYED

**Azure resources, the private dataset, live Function, public dashboard, and GitHub CI/CD are complete. The final section records the required URLs and deployment evidence.**

## Phase 2 objectives satisfied

- Deployable serverless analysis through an Azure Function App.
- Cloud dataset storage through a private Azure Blob container.
- Browser dashboard with more than the required three visualizations.
- Frontend-to-backend integration with filters, refresh, and execution metadata.
- Cloud-native resource settings, monitoring, security headers, and CI/CD workflows.

# Solution architecture

The design separates presentation, compute, storage, telemetry, and deployment concerns. The static frontend never receives an Azure Storage credential. It calls a read-only HTTP endpoint, and only the Function App crosses the private storage boundary.

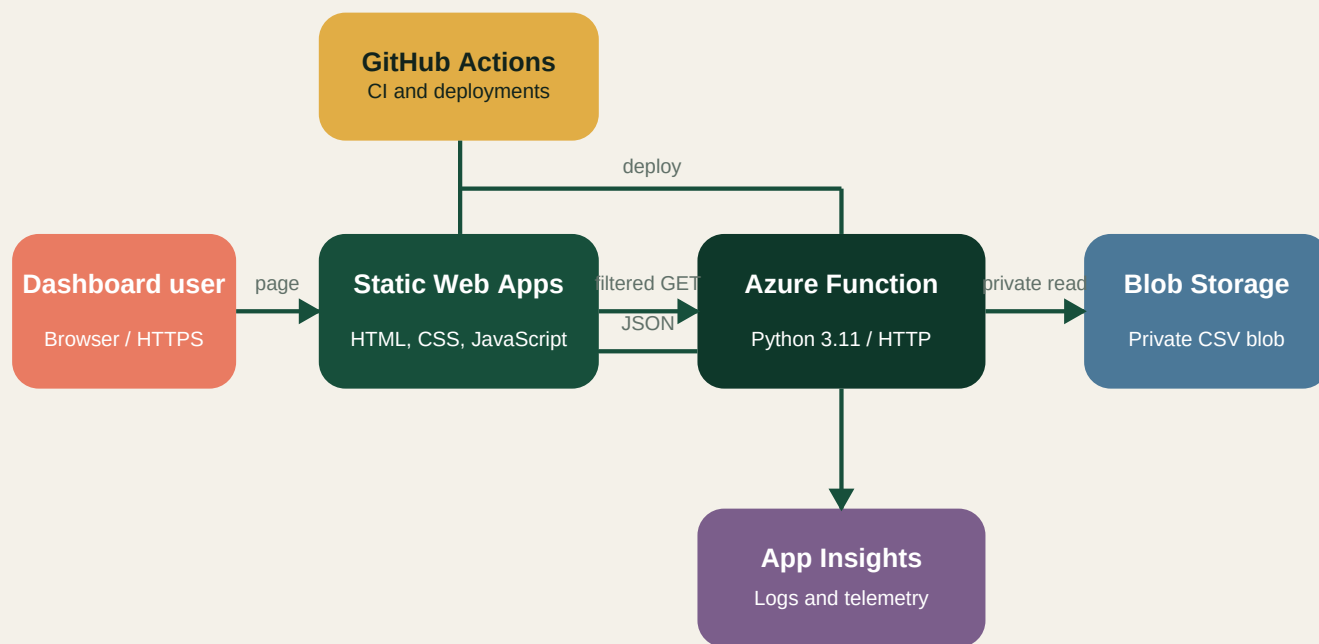


Figure 1. Phase 2 Azure architecture and primary request path.

## Request flow

1. Azure Static Web Apps serves the dashboard's HTML, CSS, JavaScript, and security configuration.
2. The dashboard requests `/api/diet-insights` with optional diet, cuisine, search, and refresh parameters.
3. The Function downloads `datasets/All_Diets.csv` from private Blob Storage or reuses a five-minute in-memory cache.
4. The shared analysis module validates required fields, normalizes text, parses numeric macros, and mean-imputes missing values.
5. Filtered metrics and four chart datasets are returned as JSON with request, source, version, and execution metadata.
6. Canvas renders redraw the visualizations and update the accessible data table without reloading the page.

## Azure services

| Service                     | Responsibility    | Configuration                                                |
|-----------------------------|-------------------|--------------------------------------------------------------|
| <b>Azure Functions</b>      | HTTP analysis API | Python 3.11, Flex Consumption, anonymous read-only trigger   |
| <b>Blob Storage</b>         | Dataset source    | Private datasets container; public blob access disabled      |
| <b>Static Web Apps</b>      | Public frontend   | Free tier, static build skipped, production API URL injected |
| <b>Application Insights</b> | Operations        | Request sampling, errors, execution telemetry                |

# Backend API and analysis

The backend uses the decorator-based Azure Functions Python v2 model. The global **function\_app.py** defines one HTTP trigger and delegates all dataset work to testable modules. Production dependencies are intentionally limited to **azure-functions** and **azure-storage-blob**; aggregation uses Python's standard library to reduce deployment size and cold-start overhead.

## Endpoint

**GET****/api/diet-insights?diet\_types=Vegan&cuisine=Indian&search=curry&refresh=true**

## Response contract

| Object          | Contents                                                                                       |
|-----------------|------------------------------------------------------------------------------------------------|
| <b>summary</b>  | Recipe count, represented diet types, and mean protein/carbs/fat                               |
| <b>filters</b>  | Available diet and cuisine values plus the applied request values                              |
| <b>charts</b>   | Average macros, distribution, protein-versus-carbs points, and top cuisines                    |
| <b>metadata</b> | UTC time, source, record counts, execution milliseconds, request ID, cache status, API version |

## Data preparation

- Required columns are resolved through normalized aliases, preserving compatibility with the Phase 1 dataset and common header variations.
- Diet and cuisine labels are whitespace-normalized and title-cased; recipe names retain readable capitalization.
- Protein, carbohydrates, and fat are parsed defensively. Missing or invalid numeric values use the column mean, then zero only if a column contains no valid values.
- Scatter results are bounded to the top 30 protein-rich recipes per represented diet, keeping HTTP responses and browser rendering compact.

## Reliability and observability

Input/data errors return HTTP 400, cloud configuration problems return 503, and unexpected exceptions return a non-sensitive 500 response. Every response includes a request ID; the Function records details in Application Insights. A bounded cache (default 300 seconds) reduces repeated Blob downloads while the refresh control can force a reload.

# Interactive dashboard

The provided Phase 2 HTML mockup was rebuilt as a complete, responsive interface with no external UI or chart dependency. Native canvas renderers keep the deployed site self-contained and avoid CDN failures. The frontend handles loading, timeout, API errors, empty results, source refresh, and responsive redraws.

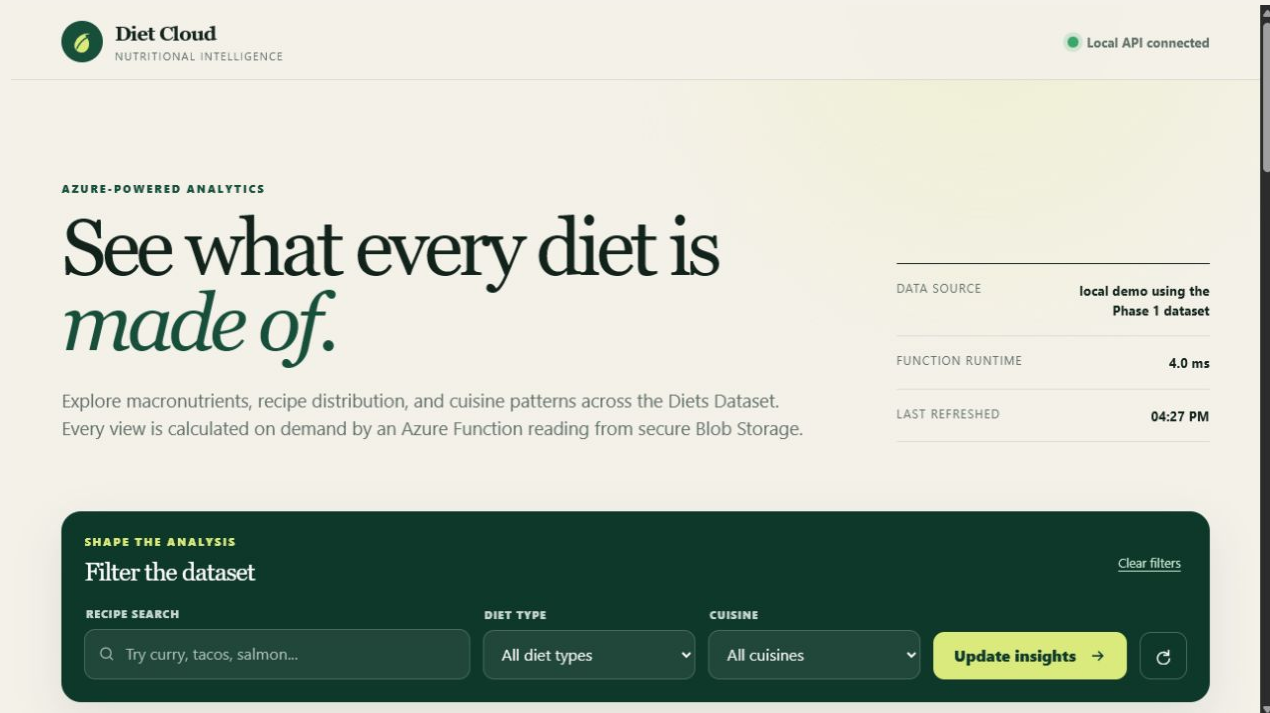


Figure 2. Local end-to-end dashboard verification using the complete Phase 1 dataset. The source label and connection status explicitly identify local mode.

## Interaction controls

- Recipe search performs a case-insensitive substring match across recipe, diet, and cuisine.
- Diet and cuisine selectors are populated from API-provided filter metadata.
- Update insights sends the selected filters; clear resets all controls; refresh bypasses the Function cache.
- Five KPI cards and the hero metadata update from the same API response as the charts.

## Accessibility

Semantic regions, headings, labels, focus states, reduced-motion support, canvas alternative labels, and a live average-macros table provide a navigable experience beyond the visual charts.

# Data visualizations

The dashboard exceeds the brief’s requirement for three distinct and meaningful visualizations. Every chart is redrawn from the filtered Azure Function response, so all views remain consistent with the KPI cards and table.

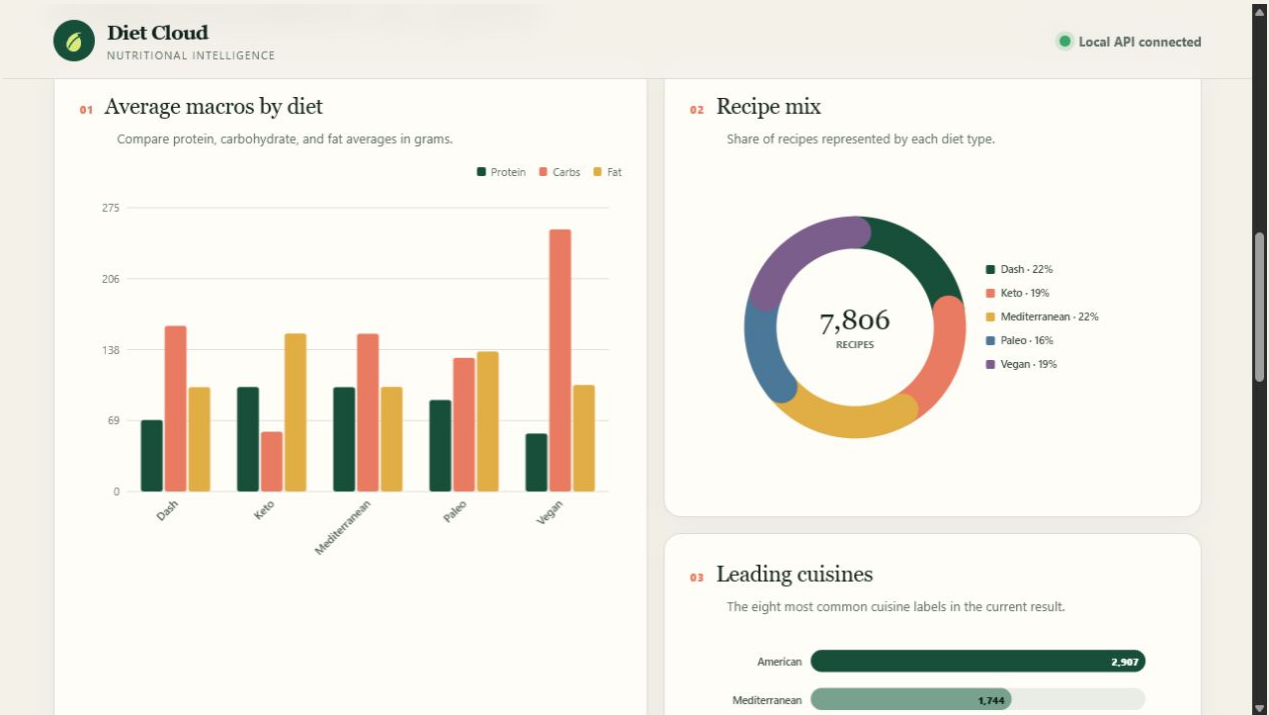


Figure 3. Verified grouped bar, donut, and leading-cuisine visualizations.

| Visualization  | Question answered                                           | Encoding                                                   |
|----------------|-------------------------------------------------------------|------------------------------------------------------------|
| Grouped bar    | How do mean macros compare across diet types?               | Diet on x-axis; grams on y-axis; color by macro            |
| Donut          | What share of recipes belongs to each diet?                 | Arc length and percentage by diet type                     |
| Horizontal bar | Which cuisines appear most often in the result?             | Cuisine label and recipe count                             |
| Scatter        | How do protein and carbs relate among protein-rich recipes? | Carbs on x, protein on y, fat by point size, diet by color |

## Interpretation example

In the complete dataset view, Vegan recipes have the highest mean carbohydrates (254.00 g), while Keto has the lowest (57.97 g). Keto and Mediterranean have similar mean protein values (101.27 g and 101.11 g). These are descriptive dataset aggregates and should not be interpreted as medical or dietary advice.

# Cloud deployment and practices

The repository includes a Bicep template and PowerShell orchestrator. The Bicep template compiled successfully with Azure Bicep CLI 0.45.15. It provisions a Python Flex Consumption Function App, two private Blob containers, a Free Static Web App, Application Insights, and Log Analytics in one resource group.

## Configuration model

| Setting                        | Purpose                 | Example                              |
|--------------------------------|-------------------------|--------------------------------------|
| DIET_STORAGE_CONNECTION_STRING | Private dataset access  | Stored only in Function App settings |
| DATA_CONTAINER_NAME            | Blob container          | datasets                             |
| DATA_BLOB_NAME                 | Dataset blob            | All_Diets.csv                        |
| CORS_ALLOWED_ORIGIN            | Restrict browser origin | Generated Static Web App URL         |
| CACHE_TTL_SECONDS              | Blob-read cache         | 300                                  |

## CI/CD

- **ci.yml** installs backend dependencies, compiles Python, runs unit tests, and verifies required frontend assets.
- **deploy-function.yml** uses the documented Azure Functions action with Flex Consumption and remote build settings.
- **deploy-frontend.yml** injects the Function API base URL from a GitHub secret, then deploys static files through the Azure Static Web Apps action.

## Security choices

Public blob access is disabled; HTTPS and TLS 1.2 are required; FTPS is disabled; CORS is scoped to the Static Web App; security headers include CSP, referrer policy, MIME sniffing protection, and restricted browser permissions. GitHub credentials are referenced only through encrypted secrets. The read-only course API is anonymous so no Function key is exposed in browser code.

## Production hardening recommendation

For a long-lived production environment, replace storage connection strings with managed identity/RBAC and, where secrets remain necessary, Key Vault references. The course template keeps a shared-key connection only to simplify one-command provisioning and the initial dataset upload.

# Verification and quality assurance

Testing covered analysis correctness, API behavior, browser interactions, responsive layout, syntax, configuration, and infrastructure compilation. Verification used the same 7,806-row dataset included in the Phase 1 archive.

| Check              | Result | Observed evidence                                                        |
|--------------------|--------|--------------------------------------------------------------------------|
| Unit tests         | PASS   | 4/4: summary, filters/search, mean imputation, required-column rejection |
| Full API           | PASS   | 7,806 recipes, 5 diet types, 4 chart datasets                            |
| Filter interaction | PASS   | Vegan + curry returned 42 recipes and 1 diet type                        |
| Clear interaction  | PASS   | Restored 7,806 recipes and cleared both controls                         |
| Browser console    | PASS   | No warning or error messages                                             |
| Mobile layout      | PASS   | 375 px client and scroll widths; no horizontal overflow                  |
| Bicep compile      | PASS   | Azure Bicep CLI 0.45.15 produced ARM JSON successfully                   |

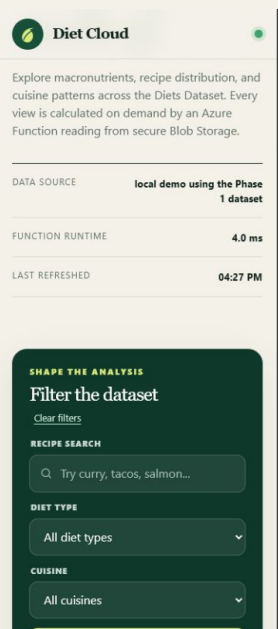


Figure 4. Mobile-width verification. Controls stack without horizontal overflow.



# Challenges, decisions, and team contributions

## Key implementation decisions

- **Phase 1 was not an HTTP Function:** the local Azurite processor was separated into reusable analysis and data-source modules, then exposed through a Python v2 HTTP trigger.
- **The supplied UI was a static shell:** chart rendering, API calls, filters, error states, responsiveness, metadata, and accessibility were implemented from scratch while retaining its nutrition-dashboard intent.
- **Cloud cost and complexity:** Flex Consumption and Static Web Apps Free were selected to minimize idle cost while preserving a current Azure architecture.
- **Credential boundaries:** storage credentials remain server-side; the frontend receives only an API base URL and aggregated JSON.
- **Dependency reliability:** canvas charts are native JavaScript, eliminating the Tailwind and chart CDN dependencies in the supplied mockup.

## Team contribution plan

| Team member         | Primary presentation ownership                               | Recommended live-demo segment                        |
|---------------------|--------------------------------------------------------------|------------------------------------------------------|
| Asad Arif           | Project framing, architecture, deliverable coordination      | Open dashboard, explain KPIs and request flow        |
| Can Ozveren         | Function API, cleaning, Blob Storage configuration           | Show endpoint JSON, filters, and private blob        |
| Muhammad Talha Arif | Frontend deployment, CI/CD, verification and cloud practices | Show GitHub Actions, responsive view, and monitoring |

## Deployment scope and limitations

Evidence reflects the live course deployment completed on July 17, 2026. Secrets, connection strings, deployment tokens, and publish profiles are intentionally excluded. Local screenshots earlier in the report document development testing; Figures 5-8 show the final Azure and GitHub deployment.

## Conclusion

The Phase 2 implementation is complete at the code, infrastructure, deployment, testing, and documentation layers. The Azure Function reads the private cloud dataset, the public Static Web App consumes its live API, and GitHub Actions validates and deploys both application layers. The project satisfies the dashboard, visualization, integration, cloud-practice, and presentation requirements.

# Final deployment evidence and submission

The account-bound deployment is complete. These public values and screenshots document the final Azure and GitHub implementation.

| Required deliverable | Final value                                                                                                                                               |
|----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| Azure Function URL   | <a href="https://dietdashasadarif2601-func.azurewebsites.net/api/diet-insights">https://dietdashasadarif2601-func.azurewebsites.net/api/diet-insights</a> |
| Dashboard URL        | <a href="https://zealous-field-0bdb4a910.7.azurestaticapps.net">https://zealous-field-0bdb4a910.7.azurestaticapps.net</a>                                 |
| GitHub repository    | <a href="https://github.com/AsadSait1998/diet-cloud-dashboard-phase2">https://github.com/AsadSait1998/diet-cloud-dashboard-phase2</a>                     |

## Evidence screenshots included

1. Resource group Overview listing Function App, Storage, Static Web App, and Application Insights.
2. Storage browser showing **datasets/All\_Diets.csv** without keys or connection strings.
3. Public dashboard showing all chart cards and **Live Azure API connected**.
4. GitHub Actions page showing successful CI, Function, and frontend workflows.

## Final verification

- Function endpoint returned HTTP 200 with summary, filters, four chart datasets, and execution metadata.
- Public dashboard connected to the live Azure API and rendered all interactive controls and charts.
- Public GitHub repository contains the frontend, backend, infrastructure, workflows, documentation, and no committed secrets.
- Final PDF contains all three public URLs and the required Azure/GitHub evidence.
- Submit the PDF, links, and repository before the course deadline.

### SUBMISSION NOTE

The repository includes **DEPLOYMENT\_GUIDE.md** and **SUBMISSION\_CHECKLIST.md** with reproducible deployment, verification, and troubleshooting guidance.

## Official references

1. Microsoft Learn - Azure Functions Python developer reference
2. Microsoft Learn - Azure Functions app settings reference
3. Microsoft Learn - Create Flex Consumption resources with Bicep
4. Microsoft Learn - Azure Functions GitHub Actions deployment
5. Microsoft Learn - Azure Static Web Apps build configuration
6. Microsoft Learn - Configure Azure Static Web Apps

# Azure deployment evidence

The following screenshots were added after the account-bound deployment steps were completed.

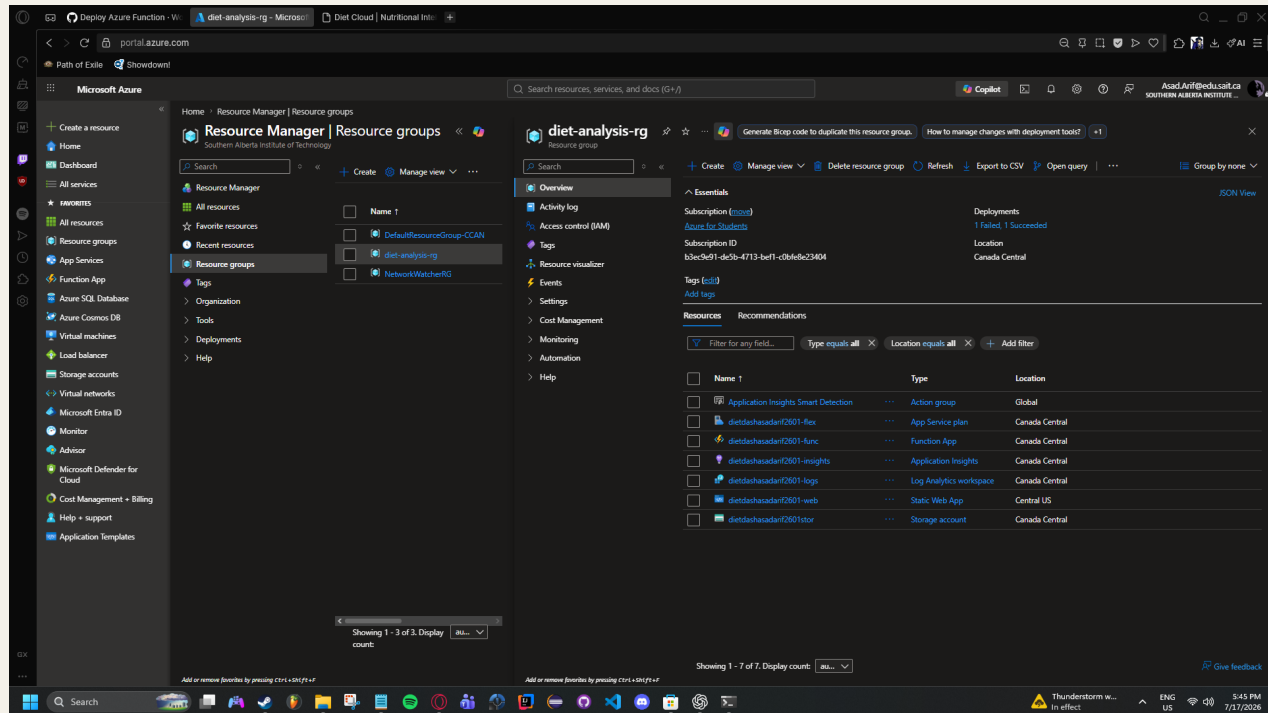


Figure 5. Azure resource group Overview showing the provisioned Phase 2 services.

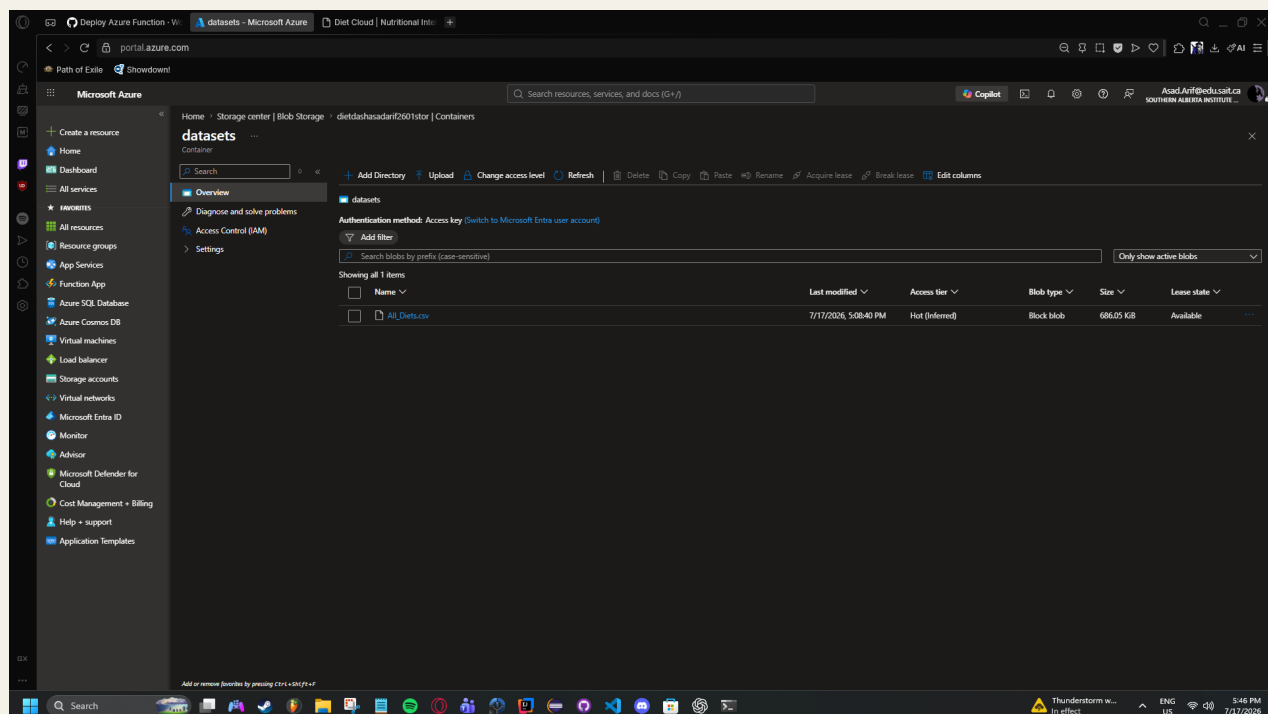


Figure 6. Private datasets container showing All\_Diets.csv without credentials.

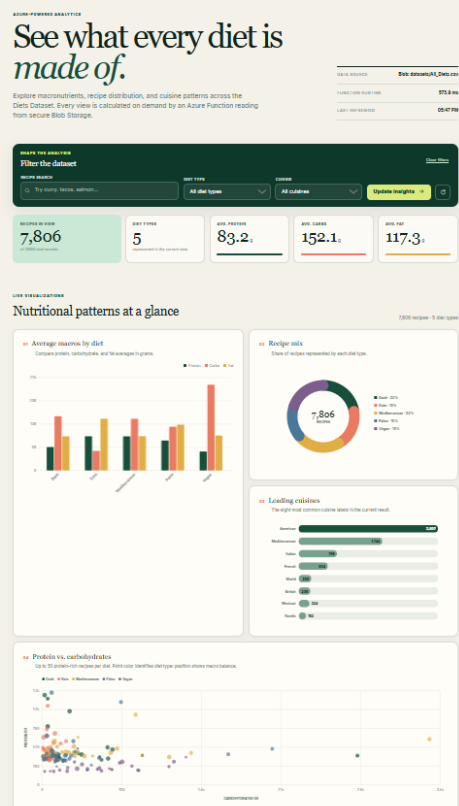


Figure 7. Public Azure Static Web App connected to the deployed Function API.

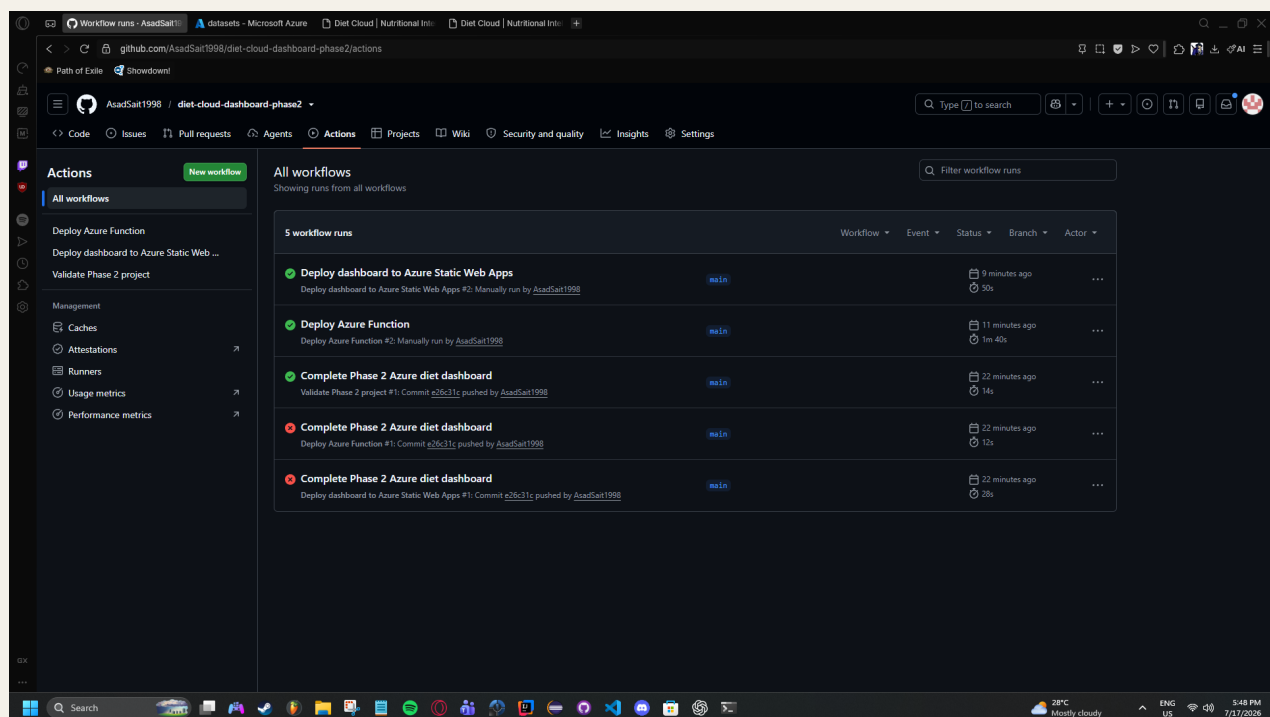


Figure 8. Successful validation, Function deployment, and frontend deployment workflows.